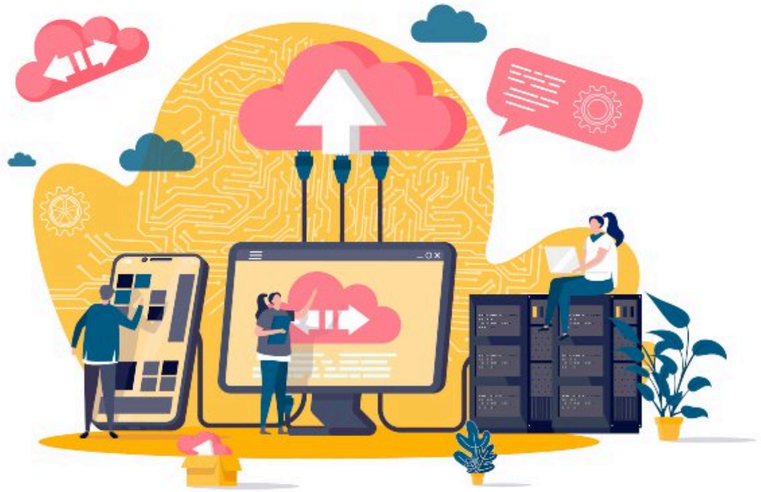
Desarrollo y Arquitectura de Software

Introducción a bases de datos + desarrollo de software



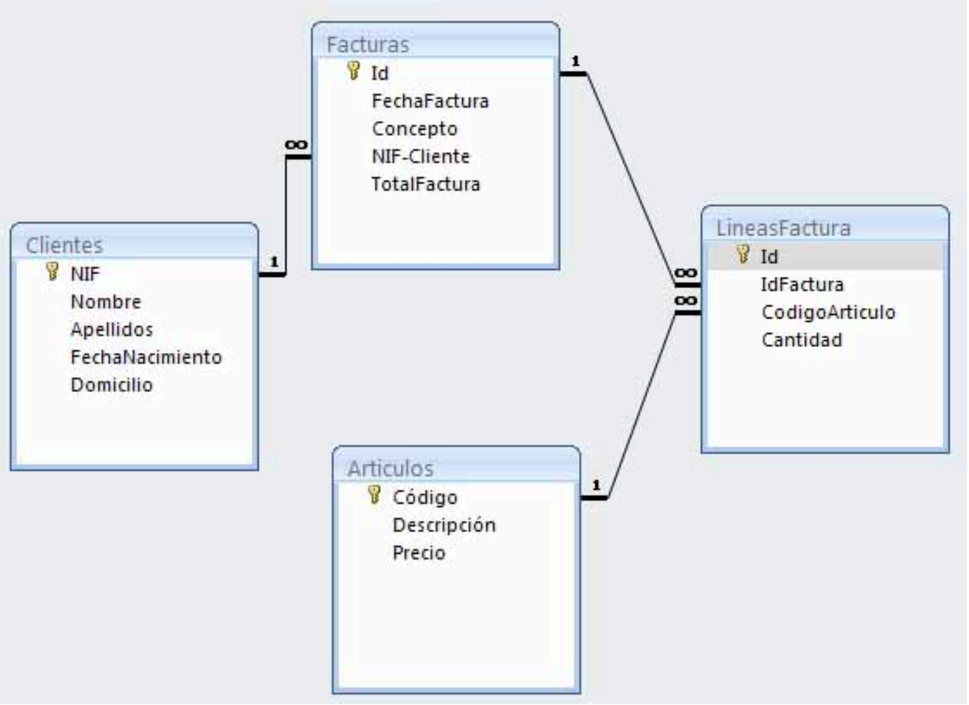


Base de datos

Una **base de datos** es un repositorio digital para almacenar, gestionar y proteger colecciones organizadas de datos.
Puede contener cualquier tipo de datos, incluidos palabras, números, imágenes, vídeos y archivos.
Se puede usar un software denominado sistema de administración de bases de datos para almacenar, recuperar y editar datos.

Base de datos relacionales

Una base de datos **relacional** es un tipo de base de datos que organiza los datos en filas y columnas, que colectivamente forman una tabla donde los puntos de datos están relacionados entre sí.



Modo conectado

Decimos que un entorno está conectado cuando el modelo de datos está conectado continuamente al motor de la base de datos.

Esto tiene algunas ventajas:

- El entorno es más fácil de mantener
- La concurrencia se controla más fácilmente
- Es más probable que los datos estén más actualizados que en otros escenarios

ADO.NET

ADO.NET es un conjunto de componentes del software que pueden ser usados por los programadores para acceder a datos y a servicios de datos.

Es comúnmente usado por los programadores para acceder y para modificar los datos almacenados en un Sistema Gestor de Bases de Datos Relacionales.

ADO.NET representa la relación entre todos los objetos que permiten la conexión a las bases datos. Está constituido por algunos objetos que conforman la arquitectura y otros que, fuera de ella, son utilizados para fines determinados.

ADO.NET

Dentro de ADO.NET tenemos los objetos **Conection, Command, dataReader**.

Si trabajamos con SQL SERVER, los renombramos a:

- **sqlConection:** crea y administra el canal de comunicación físico entre tu aplicación y la base de datos.

- **sqlCommand:** ejecuta una orden o consulta de SQL (como SELECT, INSERT, UPDATE, DELETE) o un procedimiento almacenado.

- **sqldataReader:** lee los datos de forma rápida, secuencial y de solo lectura directamente desde la base de datos.

Para poder incorporar estos objetos al código, debemos usar el espacio:
Microsoft.data.SqlClient

SqlConection

El objeto SqlConection necesita lo que llamamos **string de conexión** (una estructura de datos que contiene toda la información que usará el proveedor de datos para conectarse a la base de datos)

Data Source=[servidor]; Initial Catalog=[Nombre de BD]; Integrated Security=True;



Dicho string de conexión es asignado al objeto mediante la propiedad **ConnectionString**.

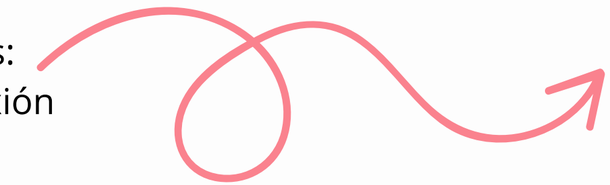
Una vez asignada la propiedad, utilizamos los métodos **Open** y **Close** para abrir y cerrar la conexión respectivamente.

SqlCommand

Este objeto se complementa con el objeto de conexión y tiene la responsabilidad de ejecutar todas las órdenes necesarias para tomar los datos de la base. Puede hacer operaciones de agregar, borrar, modificar o seleccionar.

Se deben respetar los siguientes pasos:

1. Relacione el comando con la conexión
2. Especifique el tipo de comando
3. Facilitar el comando a ejecutar



```
SqlConnection conn = new SqlConnection();

SqlCommand command = new SqlCommand();
command .Connection = conn ;
command .CommandType = CommandType.Text;
command .CommandText = "select * from Pintor";
```

Luego, se ejecuta el comando con cualquiera de las siguientes formas:

- **ExecuteReader()**: Ejecutar el comando y devolver un conjunto de datos
- **ExecuteNonQuery()**: Ejecutar el comando y no devolver nada

SqlDataReader

El objeto dataReader representa una tabla. Es una colección que recorre registro por registro.

Tiene un método llamado **Read** que lee el registro y pasa al puntero siguiente.

El **dataReader** necesita del **SqlCommad**, es decir, al ejecutar el método **ExecuteReader** del objeto **sqlComand** obtenemos un objeto **dataReader**.

La lectura de valores en un Data Reader es por su ubicación en la colección o por su nombre: ***reader.item[“Nombre”]***

```
SqlDataReader reader = command.ExecuteReader();
while ( reader .Read() )
{ .... }
reader .Close();
```

Pasos para implementar el modo conectado

01.

Declarar e instanciar
un objeto SqlConnection para determinar que origen de datos se va a utilizar.

02.

Abrir la conexión
y establecer como se va a interactuar con los datos, mediante las propiedades del objeto SqlCommand

03.

Ejecutar
los distintos comandos que fueron declarados y por último cerrar la conexión.